GenAI – Lab Observation

Name: - Spandana Basava Kulal

Reg No: - 2023BCSE07AED441

Section: - AIML-E

Date of Submission: - 16/10/2026

Index

S.No	Experiment	Date	Signature
1.	Text Processing Using Hugging Face Transformers	17/08/2026	
2.	Implementation of a Variational Autoencoder (VAE) for MNIST Handwritten Digit Generation Using TensorFlow.	24/08/2026	
3.	Implementation of Generative Adversarial Network (GAN) for MNIST Handwritten Digit Generation Using TensorFlow and Keras	8/09/2026	
4.	Implementation of Deep Convolutional Generative Adversarial Network (DCGAN) for Image Generation Using TensorFlow and Keras	22/09/2026	
5.	Implementation of Retrieval-Augmented Generation (RAG) Using LangChain, Ollama, and Hugging Face Embeddings	29/09/2026	
6.	Implementation of Retrieval-Augmented Generation (RAG) Using LangChain, Ollama, and ChromaDB	13/10/2026	

Exp No: - 1

Date: - 17/08/2026

Text Processing Using Hugging Face Transformers

Brief Description

The experiment demonstrates the use of the Hugging Face Transformers library for Natural Language Processing (NLP). Three tasks are implemented:

- Sentiment Analysis: Classifies text as positive or negative using a pretrained DistilBERT model.
- Tokenization: Splits a sentence into individual tokens and converts them into numerical token IDs.
- Text Generation: Generates new text from an input prompt using the pretrained DistilGPT-2 language model.

Program

Program 1: Sentiment Analysis

```
from transformers import pipeline
classifier = pipeline("sentiment-analysis")
texts = [
    "I love this movie.",
    "This is an excellent product.",
    "I hate this movie.",
    "This product is terrible.",
    "The movie is okay."
]
results = classifier(texts)
for text, result in zip(texts, results):
    print("Text:", text)
    print("Prediction:", result)
    print()
```

Result

```
Text: I love this movie.
Prediction: {'label': 'POSITIVE', 'score': 0.9998736381530762}

Text: This is an excellent product.
Prediction: {'label': 'POSITIVE', 'score': 0.9998733997344971}

Text: I hate this movie.
Prediction: {'label': 'NEGATIVE', 'score': 0.9996869564056396}

Text: This product is terrible.
Prediction: {'label': 'NEGATIVE', 'score': 0.9997157454490662}

Text: The movie is okay.
Prediction: {'label': 'POSITIVE', 'score': 0.9998489618301392}
```

Program 2: Tokenization

```
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained(
    "distilbert-base-uncased"
)
text = "Artificial Intelligence is changing education."
tokens = tokenizer.tokenize(text)
token_ids = tokenizer.convert_tokens_to_ids(tokens)
print("Original Text:", text)
print("\nTokens:")
for i, token in enumerate(tokens):
    print(i, "->", token)
print("\nToken IDs:")
for token, token_id in zip(tokens, token_ids):
    print(token, "->", token_id)
```

Result

```
tokenizer_config.json:   0%|          | 0.00/48.0 [00:00<?, ?B/s]
vocab.txt:   0%|          | 0.00/232k [00:00<?, ?B/s]
tokenizer.json:   0%|          | 0.00/466k [00:00<?, ?B/s]
Original Text:
Artificial Intelligence is changing education.

Tokens:
0 -> artificial
1 -> intelligence
2 -> is
3 -> changing
4 -> education
5 -> .

Token IDs:
artificial -> 7976
intelligence -> 4454
is -> 2003
changing -> 5278
education -> 2495
. -> 1012
```

Program 3: Text Generation

```
from transformers import AutoTokenizer, AutoModelForCausalLM
tokenizer = AutoTokenizer.from_pretrained("distilgpt2")
model = AutoModelForCausalLM.from_pretrained("distilgpt2")
text = "Artificial Intelligence is"
inputs = tokenizer(text, return_tensors="pt")
outputs = model.generate(
   **inputs,
   max_new_tokens=30
)
generated_text = tokenizer.decode(
   outputs[0],
   skip_special_tokens=True
)
print("Input:")
print(text)
print("\nGenerated Text:")
print(generated_text)
```

Result

```
tokenizer_config.json:   0%|          | 0.00/26.0 [00:00<?, ?B/s]
vocab.json:   0%|          | 0.00/1.04M [00:00<?, ?B/s]
merges.txt:   0%|          | 0.00/456k [00:00<?, ?B/s]
tokenizer.json:   0%|          | 0.00/1.36M [00:00<?, ?B/s]
model.safetensors: reconstructing file:   0%|          |  0.00B /  353MB
model.safetensors: downloading bytes:          |  0.00B
Loading weights:   0%|          | 0/76 [00:00<?, ?it/s]
generation_config.json:   0%|          | 0.00/124 [00:00<?, ?B/s]
Input:
Artificial Intelligence is

Generated Text:
Artificial Intelligence is a new technology that is being developed by the University of California at Berkeley.
```

Conclusion

The experiment successfully demonstrated three important NLP applications using Hugging Face Transformers: sentiment analysis, tokenization, and text generation. Pretrained Transformer models can classify text, convert natural language into numerical representations, and generate coherent text. These techniques form the foundation for advanced applications such as chatbots, document analysis, and generative AI systems.

Exp No: - 2

Date: - 24/08/2026

Implementation of a Variational Autoencoder (VAE) for MNIST Handwritten Digit Generation Using TensorFlow

Brief Description

The experiment implements a Variational Autoencoder (VAE), a type of generative deep learning model, using TensorFlow and Keras. The MNIST dataset, containing 28 × 28 grayscale images of handwritten digits, is used to train the model. The encoder compresses each image into a low-dimensional latent representation, and the decoder reconstructs images from the latent vectors. After training, the decoder generates new handwritten digit images by sampling random latent vectors.

Program

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
# Parameters
INPUT_DIM = 784
LATENT_DIM = 2
HIDDEN_DIM = 256
BATCH_SIZE = 128
EPOCHS = 20
# Load MNIST dataset
(x_train, y_train), (x_test, y_test) = (
    tf.keras.datasets.mnist.load_data())
# Normalize and flatten images
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
x_train = x_train.reshape(-1, INPUT_DIM)
x_test = x_test.reshape(-1, INPUT_DIM)
```

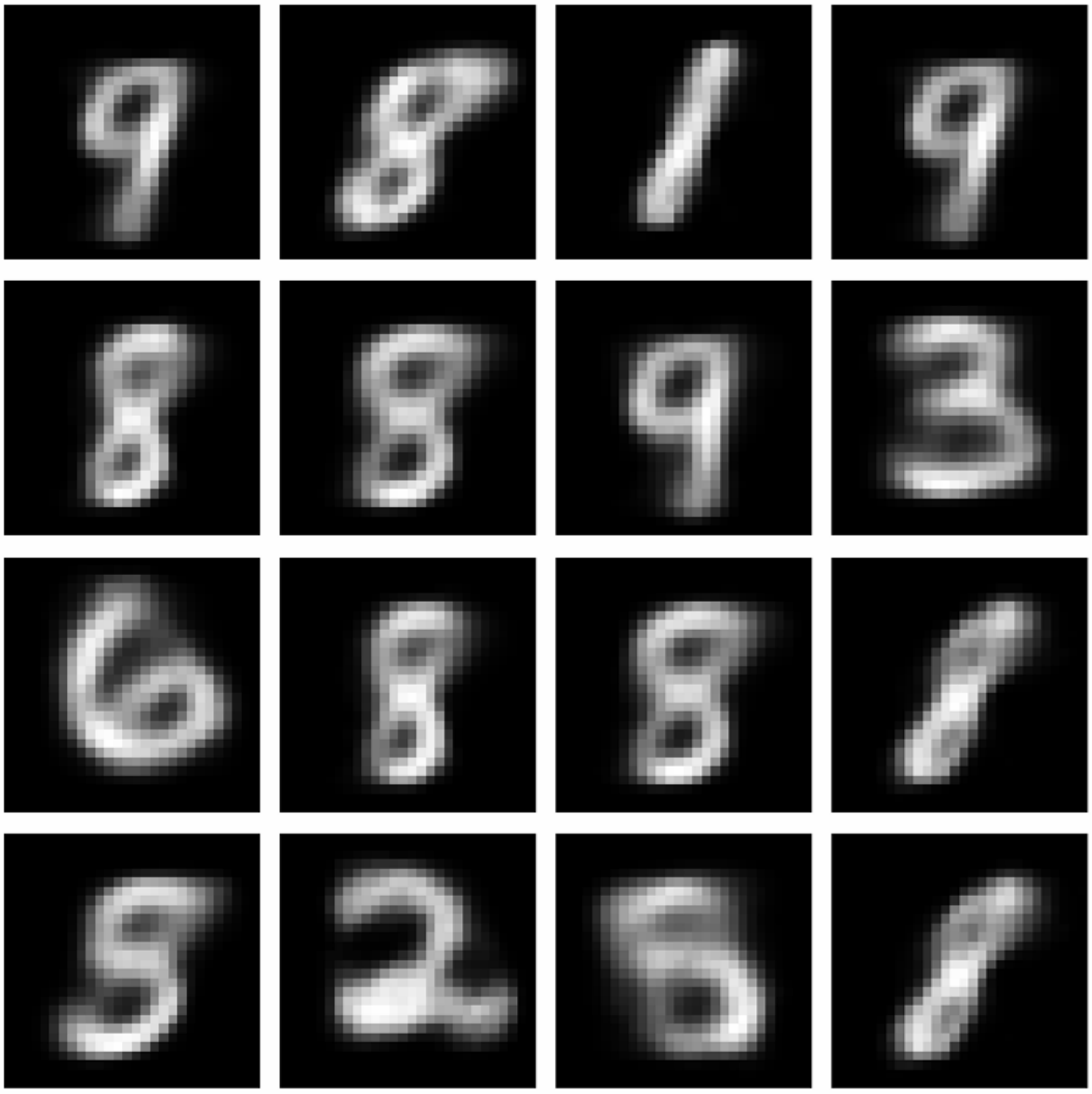
```
# Create TensorFlow dataset
train_dataset = tf.data.Dataset.from_tensor_slices(x_train)
train_dataset = train_dataset.shuffle(60000)
train_dataset = train_dataset.batch(BATCH_SIZE)
# Encoder
encoder_inputs = tf.keras.Input(shape=(INPUT_DIM,))
x = tf.keras.layers.Dense(
    HIDDEN_DIM, activation="relu"
)(encoder_inputs)
z_mean = tf.keras.layers.Dense(LATENT_DIM)(x)
z_log_var = tf.keras.layers.Dense(LATENT_DIM)(x)
# Reparameterization trick
def sampling(args):
    z_mean, z_log_var = args
    epsilon = tf.random.normal(shape=tf.shape(z_mean))
    return z_mean + tf.exp(0.5 * z_log_var) * epsilon
z = tf.keras.layers.Lambda(sampling)(
    [z_mean, z_log_var])
encoder = tf.keras.Model(
    encoder_inputs, [z_mean, z_log_var, z])
# Decoder
latent_inputs = tf.keras.Input(shape=(LATENT_DIM,))
x = tf.keras.layers.Dense(
    HIDDEN_DIM, activation="relu")(latent_inputs)
decoder_outputs = tf.keras.layers.Dense(
    INPUT_DIM, activation="sigmoid")(x)
decoder = tf.keras.Model(
    latent_inputs, decoder_outputs)
# VAE model
class VAE(tf.keras.Model):
    def __init__(self, encoder, decoder, **kwargs):
        super().__init__(**kwargs)
        self.encoder = encoder
        self.decoder = decoder
    def train_step(self, data):
        with tf.GradientTape() as tape:
            z_mean, z_log_var, z = self.encoder(data)
```

```
            reconstruction = self.decoder(z)
            reconstruction_loss = tf.reduce_mean(
                tf.reduce_sum(
                    -(
                        data * tf.math.log(
                            reconstruction + 1e-7)
                        + (1 - data) * tf.math.log(
                            1 - reconstruction + 1e-7) ),
                    axis=1 ))
kl_loss = -0.5 * tf.reduce_mean(
                tf.reduce_sum(
                    1 + z_log_var
                    - tf.square(z_mean)
                    - tf.exp(z_log_var),
                    axis=1) )
            total_loss = reconstruction_loss + kl_loss
        gradients = tape.gradient(
            total_loss, self.trainable_weights )
        self.optimizer.apply_gradients(
            zip(gradients, self.trainable_weights))
        return {
            "loss": total_loss,
            "reconstruction_loss": reconstruction_loss,
            "kl_loss": kl_loss  }
# Create and compile VAE
vae = VAE(encoder, decoder)
vae.compile(
    optimizer=tf.keras.optimizers.Adam(
        learning_rate=0.001))
# Train the VAE
vae.fit(
    train_dataset,
    epochs=EPOCHS)
# Generate new handwritten digits
number_of_digits = 16
random_latent_vectors = tf.random.normal(
shape=(number_of_digits, LATENT_DIM))
```

```
generated_digits = decoder(random_latent_vectors)
# Display generated digits
plt.figure(figsize=(8, 8))
for i in range(number_of_digits):
    ax = plt.subplot(4, 4, i + 1)
    plt.imshow(
        generated_digits[i].numpy().reshape(28, 28),
        cmap="gray")
    plt.axis("off")
plt.tight_layout()
plt.show()
```

Result

```
Epoch 1/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 11ms/step - kl_loss: 5.0275 - loss: 179.3900 - reconstruction_loss: 174.3625
Epoch 2/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - kl_loss: 4.6337 - loss: 163.7754 - reconstruction_loss: 159.1417
Epoch 3/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - kl_loss: 4.9531 - loss: 164.4674 - reconstruction_loss: 159.5143
Epoch 4/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - kl_loss: 5.0541 - loss: 171.0349 - reconstruction_loss: 165.9809
Epoch 5/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - kl_loss: 5.0538 - loss: 165.3059 - reconstruction_loss: 160.2521
Epoch 6/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - kl_loss: 5.2698 - loss: 162.2994 - reconstruction_loss: 157.0296
Epoch 7/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - kl_loss: 5.3907 - loss: 160.2741 - reconstruction_loss: 154.8834
Epoch 8/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - kl_loss: 5.6033 - loss: 156.6125 - reconstruction_loss: 151.0092
Epoch 9/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 6s 12ms/step - kl_loss: 5.4727 - loss: 165.8742 - reconstruction_loss: 160.4015
Epoch 10/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 6s 12ms/step - kl_loss: 5.1919 - loss: 165.4955 - reconstruction_loss: 160.3036
Epoch 11/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - kl_loss: 5.6312 - loss: 158.6481 - reconstruction_loss: 153.0170
Epoch 12/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 6s 12ms/step - kl_loss: 5.6787 - loss: 152.9733 - reconstruction_loss: 147.2946
Epoch 13/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 6s 12ms/step - kl_loss: 5.6349 - loss: 149.9398 - reconstruction_loss: 144.3049
Epoch 14/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 6s 12ms/step - kl_loss: 5.7323 - loss: 152.5482 - reconstruction_loss: 146.8159
Epoch 15/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 6s 12ms/step - kl_loss: 5.8474 - loss: 149.8848 - reconstruction_loss: 144.0373
Epoch 16/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 6s 12ms/step - kl_loss: 5.6170 - loss: 158.6619 - reconstruction_loss: 153.0449
Epoch 17/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 6s 13ms/step - kl_loss: 5.7322 - loss: 156.5323 - reconstruction_loss: 150.8000
Epoch 18/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 6s 12ms/step - kl_loss: 5.8698 - loss: 155.4669 - reconstruction_loss: 149.5971
Epoch 19/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 6s 13ms/step - kl_loss: 5.7350 - loss: 147.5660 - reconstruction_loss: 141.8310
Epoch 20/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 6s 12ms/step - kl_loss: 5.9816 - loss: 152.7321 - reconstruction_loss: 146.7505
```



Conclusion

The experiment successfully implemented a Variational Autoencoder using TensorFlow and the MNIST dataset. The model learned a two-dimensional latent representation of handwritten digits and generated new digit-like images through its decoder. The experiment demonstrates the use of generative deep learning for image reconstruction and synthesis. VAEs can also be applied to image generation, representation learning, anomaly detection, and other generative AI tasks.

Exp No: - 3

Date: - 8/09/2026

Implementation of Generative Adversarial Network (GAN) for MNIST Handwritten Digit Generation Using TensorFlow and Keras

Brief Description

This experiment implements a Generative Adversarial Network (GAN) using TensorFlow and Keras to generate synthetic handwritten digit images. The MNIST dataset, containing 60,000 grayscale images of size 28 × 28 pixels, is used for training.

A GAN consists of two neural networks: the Generator, which creates fake handwritten digit images from random noise, and the Discriminator, which distinguishes real images from generated images. Both networks are trained competitively to improve the quality of the generated images.

Program

```
import tensorflow as tf
from tensorflow.keras import layers, Sequential
import numpy as np
import matplotlib.pyplot as plt
# 1. Load MNIST dataset
(x_train, y_train), (_, _) = tf.keras.datasets.mnist.load_data()
print("Original MNIST Shape:", x_train.shape)
print("Number of training images:", len(x_train))
# 2. Normalize pixel values
x_train = x_train.astype("float32")
x_train = (x_train - 127.5) / 127.5
# 3. Reshape data
x_train = np.expand_dims(x_train, axis=-1)
print("New shape:", x_train.shape)
```

```
# 4. Create dataset
BATCH_SIZE = 128
dataset = tf.data.Dataset.from_tensor_slices(x_train)
dataset = dataset.shuffle(60000).batch(BATCH_SIZE)
# 5. Build generator
LATENT_DIM = 100
def build_generator():
    model = Sequential([
        layers.Input(shape=(LATENT_DIM,)),
        layers.Dense(256),
        layers.LeakyReLU(negative_slope=0.2),
        layers.Dense(512),
        layers.LeakyReLU(negative_slope=0.2),
        layers.Dense(1024),
        layers.LeakyReLU(negative_slope=0.2),
        layers.Dense(784, activation="tanh"),
        layers.Reshape((28, 28, 1))
    ])
    return model
generator = build_generator()
# 6. Build discriminator
def build_discriminator():
    model = Sequential([
        layers.Input(shape=(28, 28, 1)),
        layers.Flatten(),
        layers.Dense(512),
        layers.LeakyReLU(negative_slope=0.2),
        layers.Dense(256),
        layers.LeakyReLU(negative_slope=0.2),
        layers.Dense(1, activation="sigmoid")])
    return model
discriminator = build_discriminator()
# 7. Define loss and optimizers
cross_entropy = tf.keras.losses.BinaryCrossentropy()
generator_optimizer = tf.keras.optimizers.Adam(
    learning_rate=0.0002, beta_1=0.5)
discriminator_optimizer = tf.keras.optimizers.Adam(
```

```
    learning_rate=0.0002, beta_1=0.5)
# 8. Training step
@tf.function
def train_step(real_images):
    batch_size = tf.shape(real_images)[0]
    noise = tf.random.normal(
        shape=(batch_size, LATENT_DIM))
    with tf.GradientTape() as gen_tape, \
         tf.GradientTape() as disc_tape:
        # Generate fake images
        fake_images = generator(noise, training=True)
        # Predict real and fake images
        real_predictions = discriminator(
            real_images, training=True)
        fake_predictions = discriminator(
            fake_images, training=True)
        # Discriminator loss
        real_loss = cross_entropy(
            tf.ones_like(real_predictions),
            real_predictions)
        fake_loss = cross_entropy(
            tf.zeros_like(fake_predictions),
            fake_predictions)
        discriminator_loss = real_loss + fake_loss
        # Generator loss
        generator_loss = cross_entropy(
            tf.ones_like(fake_predictions),
            fake_predictions)
    # Calculate gradients
    generator_gradients = gen_tape.gradient(
        generator_loss,
        generator.trainable_variables)
    discriminator_gradients = disc_tape.gradient(
        discriminator_loss,
        discriminator.trainable_variables )
    # Update network weights
    generator_optimizer.apply_gradients(
```

```
        zip(generator_gradients,
            generator.trainable_variables))
    discriminator_optimizer.apply_gradients(
        zip(discriminator_gradients,
            discriminator.trainable_variables))
    return generator_loss, discriminator_loss
# 9. Generate and display images
def generate_and_show_images(epoch, num_images=16):
    noise = tf.random.normal(
        shape=(num_images, LATENT_DIM))
    generated_images = generator(noise, training=False)
    generated_images = (generated_images.numpy() + 1) / 2
    plt.figure(figsize=(8, 8))
    for i in range(num_images):
        plt.subplot(4, 4, i + 1)
        plt.imshow(
            generated_images[i, :, :, 0],
            cmap="gray")
        plt.axis("off")
    plt.suptitle(f"Generated MNIST Digits - Epoch {epoch}")
    plt.show()
# 10. Train the GAN
EPOCHS = 10
for epoch in range(EPOCHS):
    print(f"\nEpoch {epoch + 1}/{EPOCHS}")
for real_images in dataset:
        gen_loss, disc_loss = train_step(real_images)
    print("Generator Loss:", float(gen_loss))
    print("Discriminator Loss:", float(disc_loss))
    if (epoch + 1) % 5 == 0:
        generate_and_show_images(epoch + 1)
# 11. Generate final images
print("Generating final MNIST images...")
generate_and_show_images(EPOCHS)
```

Result

```
Original MNIST shape: (60000, 28, 28)
Number of training images: 60000

Minimum pixel value: -1.0
Maximum pixel value: 1.0

New shape: (60000, 28, 28, 1)

================ GENERATOR ================
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense_7 (Dense)	(None, 256)	25,856
leaky_re_lu_5 (LeakyReLU)	(None, 256)	0
dense_8 (Dense)	(None, 512)	131,584
leaky_re_lu_6 (LeakyReLU)	(None, 512)	0
dense_9 (Dense)	(None, 1024)	525,312
leaky_re_lu_7 (LeakyReLU)	(None, 1024)	0
dense_10 (Dense)	(None, 784)	803,600
reshape_1 (Reshape)	(None, 28, 28, 1)	0

Total params: 1,486,352 (5.67 MB)
Trainable params: 1,486,352 (5.67 MB)
Non-trainable params: 0 (0.00 B)

```
============== DISCRIMINATOR ==============
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
flatten_1 (Flatten)	(None, 784)	0
dense_11 (Dense)	(None, 512)	401,920
leaky_re_lu_8 (LeakyReLU)	(None, 512)	0
dense_12 (Dense)	(None, 256)	131,328
leaky_re_lu_9 (LeakyReLU)	(None, 256)	0
dense_13 (Dense)	(None, 1)	257

Total params: 533,505 (2.04 MB)
Trainable params: 533,505 (2.04 MB)
Non-trainable params: 0 (0.00 B)

```
Epoch 1/10
Generator Loss: 1.1164157390594482
Discriminator Loss: 1.002203345298767

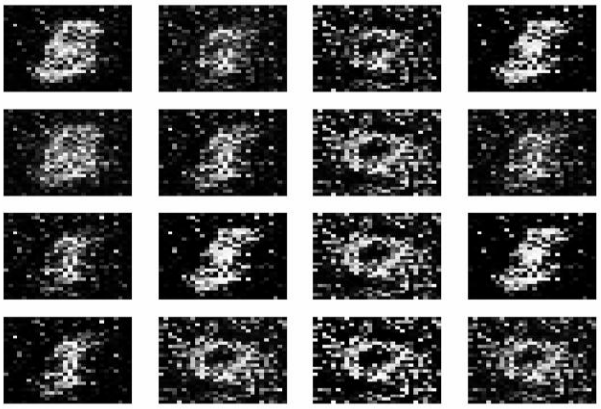
Epoch 2/10
Generator Loss: 3.1197025775909424
Discriminator Loss: 1.5606876611709595

Epoch 3/10
Generator Loss: 1.3974151611328125
Discriminator Loss: 0.8405321836471558

Epoch 4/10
Generator Loss: 0.7355871200561523
Discriminator Loss: 0.8589959144592285

Epoch 5/10
Generator Loss: 0.9896602630615234
Discriminator Loss: 0.6388436555862427
```

Generated MNIST Digits - Epoch 5



```
Epoch 6/10
Generator Loss: 1.551147460937
Discriminator Loss: 0.8168414235115051

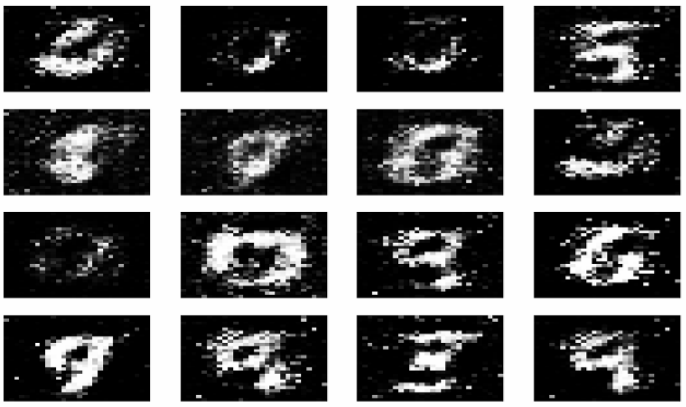
Epoch 7/10
Generator Loss: 1.582504391670227
Discriminator Loss: 0.8556318283081055

Epoch 8/10
Generator Loss: 1.5532941818237305
Discriminator Loss: 0.7286840677261353

Epoch 9/10
Generator Loss: 1.3891396522521973
Discriminator Loss: 0.7569011449813843

Epoch 10/10
Generator Loss: 1.8131380081176758
Discriminator Loss: 0.7190013527870178
```

Generated MNIST Digits - Epoch 10



```
Generating final MNIST images...
```

Conclusion

The experiment successfully implemented a Generative Adversarial Network using TensorFlow and Keras. The generator learned to produce synthetic handwritten digit images, while the discriminator learned to distinguish generated images from real images. The experiment demonstrates the fundamental principles of adversarial training and generative deep learning. The quality of generated images could be improved through additional training, architectural improvements, and techniques such as Deep Convolutional GANs (DCGANs).

Exp No: - 4

Date: - 22/09/2026

Implementation of Deep Convolutional Generative Adversarial Network (DCGAN) for Image Generation Using TensorFlow and Keras

Brief Description

This experiment implements a Deep Convolutional Generative Adversarial Network (DCGAN) to generate synthetic images using TensorFlow and Keras. The Caltech101 image dataset is used, specifically the bonsai category, containing 128 RGB images resized to \(64 \times 64\) pixels.

DCGAN consists of two neural networks: the Generator, which creates synthetic images from random noise using transposed convolutional layers, and the Discriminator, which distinguishes real images from generated images using convolutional layers. Both networks compete during training, helping the generator improve the quality of its images.

Program

```
import os
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from PIL import Image
from pathlib import Path
from tensorflow.keras import layers, Sequential
# 1. LOAD AND PREPROCESS DATASET
IMG_SIZE = 64
BATCH_SIZE = 16
LATENT_DIM = 100
EPOCHS = 100
DATA_DIR = os.path.join(
```

```
    "data", "101_ObjectCategories", "bonsai")
if not os.path.isdir(DATA_DIR):
    raise FileNotFoundError(
        f"Dataset folder not found: {DATA_DIR}. "
        "Update DATA_DIR to your bonsai folder path.")
image_paths = []
for ext in ("*.jpg", "*.jpeg", "*.png", "*.bmp"):
    image_paths.extend(Path(DATA_DIR).glob(ext))
    image_paths.extend(Path(DATA_DIR).glob(ext.upper()))
images = []
for path in sorted(set(image_paths)):
    try:
        image = Image.open(path).convert("RGB")
        image = image.resize((IMG_SIZE, IMG_SIZE))
        images.append(np.asarray(image, dtype=np.float32))
    except Exception:
        continue
if len(images) < 2:
    raise ValueError("Not enough valid images in the bonsai folder.")
data = np.array(images, dtype=np.float32)
data = (data / 127.5) - 1.0
print("Dataset shape:", data.shape)
print("Number of images:", len(data))
print("Minimum pixel value:", data.min())
print("Maximum pixel value:", data.max())
dataset = tf.data.Dataset.from_tensor_slices(data)
dataset = dataset.shuffle(len(data)).batch(BATCH_SIZE)
# 2. BUILD THE GENERATOR
def build_generator():
    model = Sequential([
        layers.Input(shape=(LATENT_DIM,)),
        layers.Dense(8 * 8 * 256, use_bias=False),
        layers.BatchNormalization(),
        layers.LeakyReLU(negative_slope=0.2),
        layers.Reshape((8, 8, 256)),
        layers.Conv2DTranspose(
            128, 4, strides=2, padding="same",
```

```
            use_bias=False),
        layers.BatchNormalization(),
        layers.LeakyReLU(negative_slope=0.2),
        layers.Conv2DTranspose(
            64, 4, strides=2, padding="same",
            use_bias=False),
        layers.BatchNormalization(),
        layers.LeakyReLU(negative_slope=0.2),
        layers.Conv2DTranspose(
            3, 4, strides=2, padding="same",
            activation="tanh") ], name="Generator")
    return model
generator = build_generator()
generator.summary()
# 3. BUILD THE DISCRIMINATOR
def build_discriminator():
    model = Sequential([
        layers.Input(shape=(IMG_SIZE, IMG_SIZE, 3)),
        layers.Conv2D(64, 4, strides=2, padding="same"),
        layers.LeakyReLU(negative_slope=0.2),
        layers.Dropout(0.3),
        layers.Conv2D(128, 4, strides=2, padding="same"),
        layers.LeakyReLU(negative_slope=0.2),
        layers.Dropout(0.3),
        layers.Conv2D(256, 4, strides=2, padding="same"),
        layers.LeakyReLU(negative_slope=0.2),
        layers.Dropout(0.3),
        layers.Flatten(),
        layers.Dense(1)], name="Discriminator")
    return model
discriminator = build_discriminator()
discriminator.summary()
# 4. DEFINE LOSS FUNCTIONS AND OPTIMIZERS
cross_entropy = tf.keras.losses.BinaryCrossentropy(
    from_logits=True)
generator_optimizer = tf.keras.optimizers.Adam(
    learning_rate=0.0002, beta_1=0.5)
```

```
discriminator_optimizer = tf.keras.optimizers.Adam(
    learning_rate=0.0002, beta_1=0.5)
# 5. DEFINE ONE TRAINING STEP
@tf.function
def train_step(real_images):
    batch_size = tf.shape(real_images)[0]
    noise = tf.random.normal(
        shape=tf.stack([batch_size, LATENT_DIM]))
    with tf.GradientTape() as gen_tape, \
         tf.GradientTape() as disc_tape:
        # Generate fake images
        fake_images = generator(noise, training=True)
        # Predict real and fake images
        real_output = discriminator(
            real_images, training=True)
        fake_output = discriminator(
            fake_images, training=True)
        # Discriminator loss
        real_loss = cross_entropy(
            tf.ones_like(real_output), real_output)
        fake_loss = cross_entropy(
            tf.zeros_like(fake_output), fake_output)
        disc_loss = real_loss + fake_loss
        # Generator loss
        gen_loss = cross_entropy(
            tf.ones_like(fake_output), fake_output)
    # Calculate gradients
    gen_gradients = gen_tape.gradient(
        gen_loss, generator.trainable_variables)
    disc_gradients = disc_tape.gradient(
        disc_loss, discriminator.trainable_variables)
    # Update model weights
    generator_optimizer.apply_gradients(
        zip(gen_gradients, generator.trainable_variables))
    discriminator_optimizer.apply_gradients(
        zip(disc_gradients, discriminator.trainable_variables))
    return gen_loss, disc_loss
```

```
# 6. GENERATE AND DISPLAY IMAGES
def generate_and_show_images(epoch, num_images=16):
    noise = tf.random.normal((num_images, LATENT_DIM))
    generated_images = generator(
        noise, training=False).numpy()
    # Convert pixel values from [-1, 1] to [0, 1]
    generated_images = (generated_images + 1.0) / 2.0
    plt.figure(figsize=(8, 8))
    for i in range(num_images):
        plt.subplot(4, 4, i + 1)
        plt.imshow(np.clip(generated_images[i], 0, 1))
        plt.axis("off")
    plt.suptitle(f"Generated Bonsai Images - Epoch {epoch}")
    plt.tight_layout()
    plt.show()
# 7. TRAIN THE DCGAN
for epoch in range(1, EPOCHS + 1):
    gen_loss = tf.constant(0.0)
    disc_loss = tf.constant(0.0)
    for real_images in dataset:
        gen_loss, disc_loss = train_step(real_images)
    print(
        f"Epoch {epoch}/{EPOCHS} | "
        f"Generator Loss: {float(gen_loss):.4f} | "
        f"Discriminator Loss: {float(disc_loss):.4f}")
    if epoch % 20 == 0:
        generate_and_show_images(epoch)
# 8. DISPLAY FINAL GENERATED IMAGES
print("Generating final bonsai images...")
generate_and_show_images(EPOCHS)
# 9. SAVE THE TRAINED MODELS
generator.save("dcgan_generator.keras")
discriminator.save("dcgan_discriminator.keras")
print("DCGAN training completed.")
print("Models saved successfully.")
```

Result



Model: "Generator"

Layer (type)	Output Shape	Param #
Generator-Hidden-Layer-1 (Dense)	(None, 8192)	827392
Generator-Hidden-Layer-Reshape-1 (Reshape)	(None, 8, 8, 128)	0
Generator-Hidden-Layer-2 (Conv2DTranspose)	(None, 16, 16, 128)	262272
Generator-Hidden-Layer-Activation-2 (ReLU)	(None, 16, 16, 128)	0
Generator-Hidden-Layer-3 (Conv2DTranspose)	(None, 32, 32, 256)	524544
Generator-Hidden-Layer-Activation-3 (ReLU)	(None, 32, 32, 256)	0
Generator-Hidden-Layer-4 (Conv2DTranspose)	(None, 64, 64, 512)	2097664
Generator-Hidden-Layer-Activation-4 (ReLU)	(None, 64, 64, 512)	0
Generator-Output-Layer (Conv2D)	(None, 64, 64, 3)	38403

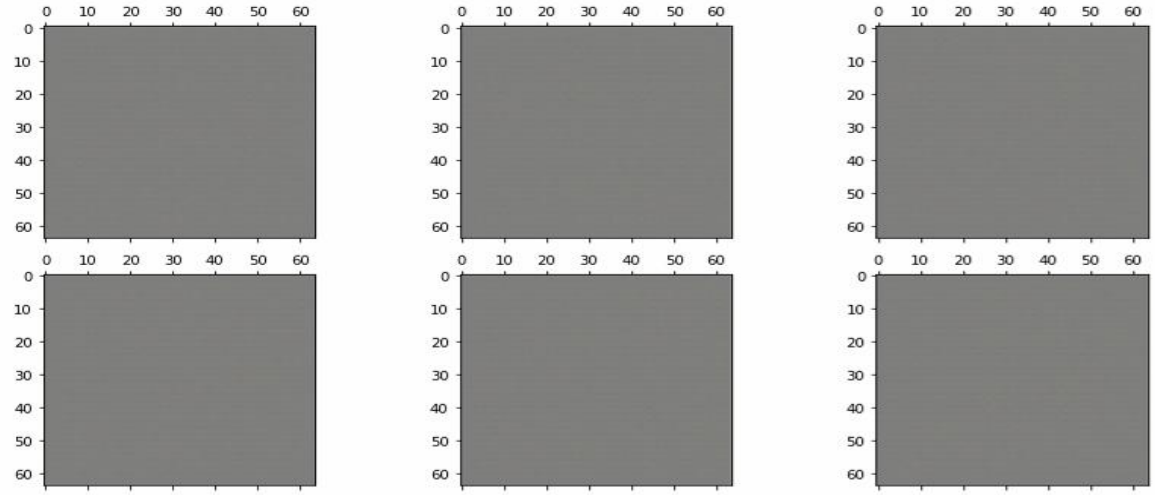
Total params: 3,750,275
Trainable params: 3,750,275
Non-trainable params: 0

Model: "Discriminator"

Layer (type)	Output Shape	Param #
Discriminator-Hidden-Layer-1 (Conv2D)	(None, 32, 32, 64)	3136
Discriminator-Hidden-Layer-Activation-1 (LeakyReLU)	(None, 32, 32, 64)	0
Discriminator-Hidden-Layer-2 (Conv2D)	(None, 16, 16, 128)	131200
Discriminator-Hidden-Layer-Activation-2 (LeakyReLU)	(None, 16, 16, 128)	0
Discriminator-Hidden-Layer-3 (Conv2D)	(None, 8, 8, 128)	262272
Discriminator-Hidden-Layer-Activation-3 (LeakyReLU)	(None, 8, 8, 128)	0
Discriminator-Flatten-Layer (Flatten)	(None, 8192)	0
Discriminator-Flatten-Layer-Dropout (Dropout)	(None, 8192)	0
Discriminator-Output-Layer (Dense)	(None, 1)	8193

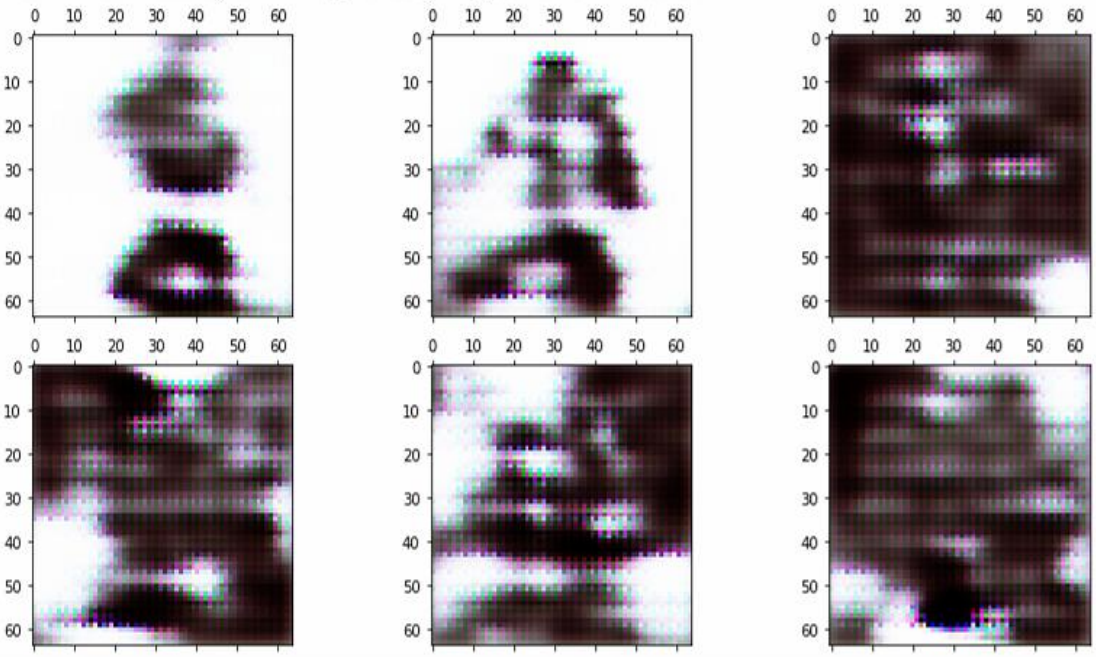
Total params: 404,801
Trainable params: 404,801
Non-trainable params: 0

```
Epoch number:  0
*** Training ***
Discriminator Loss  0.719986617565155
Generator Loss:  0.693274974822998
*** Evaluation ***
Discriminator Accuracy on REAL images:  1.0
Discriminator Accuracy on FAKE (generated) images:  0.0
```

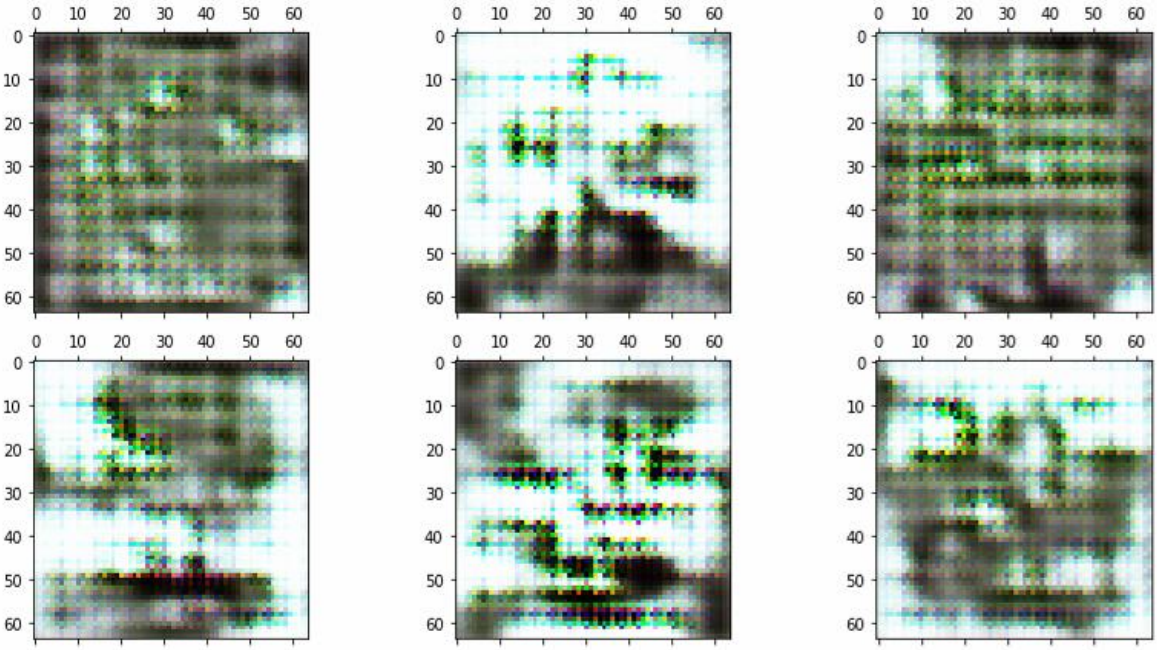


```
Epoch number:  100
*** Training ***
Discriminator Loss  0.313886821269989
Generator Loss:  3.1102733612060547
*** Evaluation ***
Discriminator Accuracy on REAL images:  0.7400000095367432
Discriminator Accuracy on FAKE (generated) images:  1.0
```

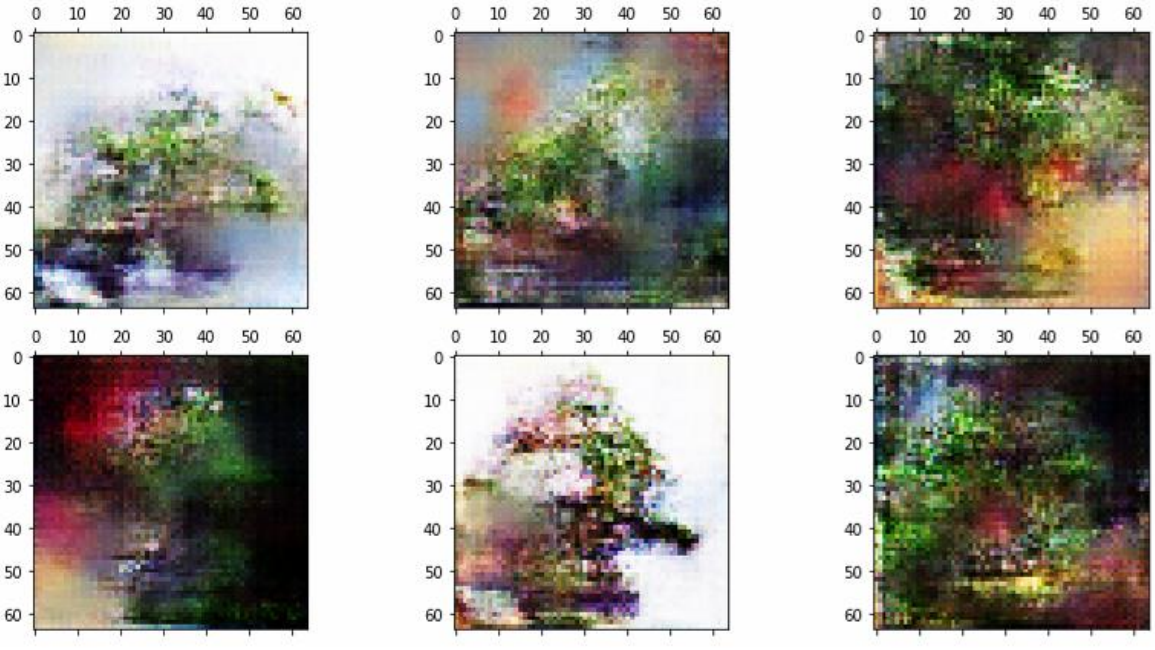
```
Epoch number:  400
*** Training ***
Discriminator Loss  0.5270434617996216
Generator Loss:  1.8168269395828247
*** Evaluation ***
Discriminator Accuracy on REAL images:  0.6000000238418579
Discriminator Accuracy on FAKE (generated) images:  0.9599999785423279
```

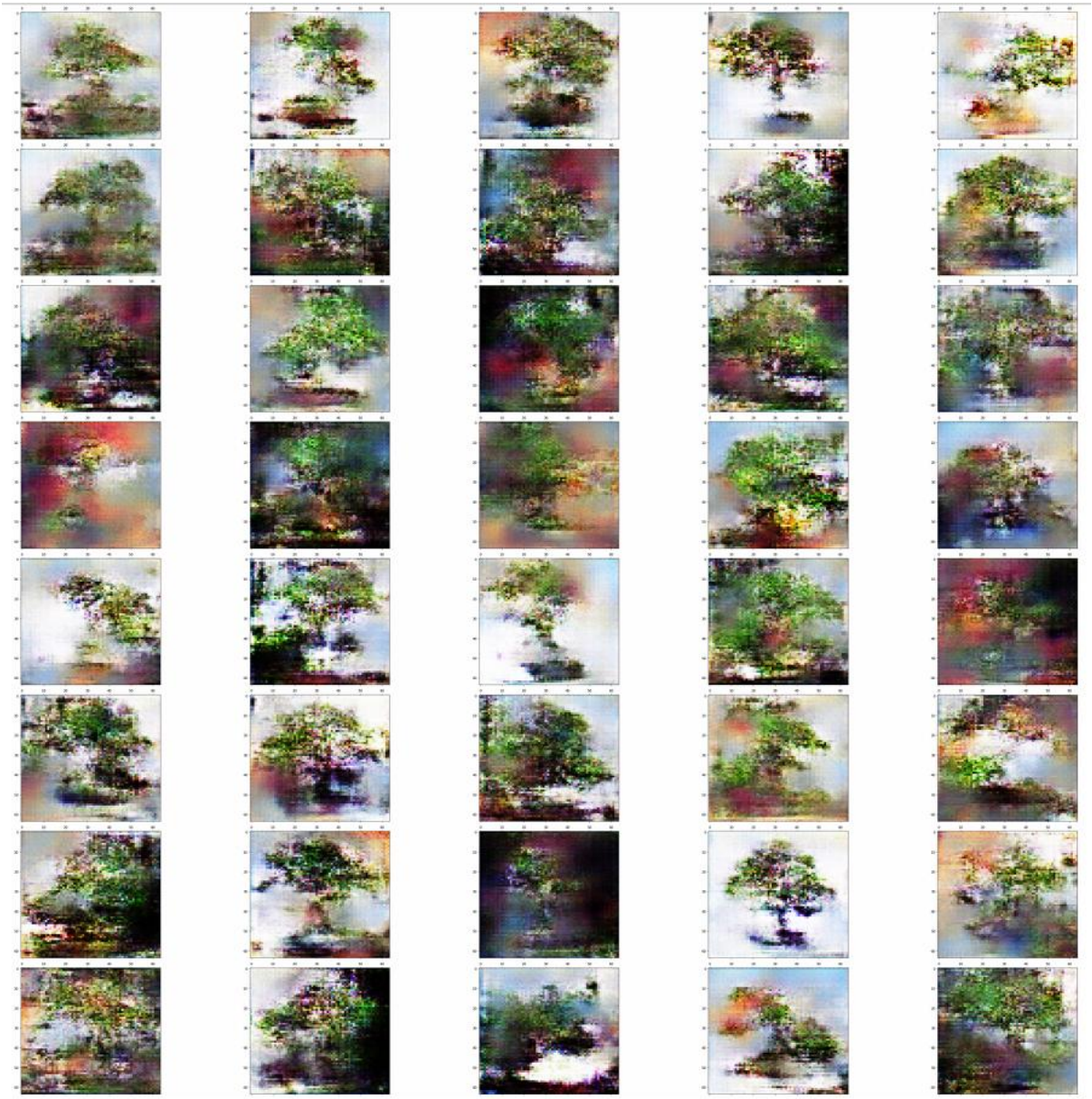


```
Epoch number:  600
*** Training ***
Discriminator Loss  0.35676681995391846
Generator Loss:  3.410188913345337
*** Evaluation ***
Discriminator Accuracy on REAL images:  0.6200000047683716
Discriminator Accuracy on FAKE (generated) images:  0.6399999856948853
```



```
Epoch number:  1000
*** Training ***
Discriminator Loss  0.41352957487106323
Generator Loss:  1.3003017902374268
*** Evaluation ***
Discriminator Accuracy on REAL images:  0.9200000166893005
Discriminator Accuracy on FAKE (generated) images:  0.9599999785423279
```





Conclusion

The experiment demonstrated the implementation of a DCGAN using TensorFlow and Keras for generating synthetic bonsai images from the Caltech101 dataset. The generator learned basic image features through adversarial training, while the discriminator learned to distinguish real images from generated ones. Although the model produced recognizable bonsai-like images, the results contained noise and distortions. Image quality could be improved by increasing the dataset size, applying data augmentation, tuning the network architecture, and using techniques such as batch normalization and improved GAN training strategies.

Exp No: - 5

Date: - 29/09/2026

Implementation of Retrieval-Augmented Generation (RAG) Using LangChain, Ollama, and Hugging Face Embeddings

Brief Description

This experiment demonstrates a basic Retrieval-Augmented Generation (RAG) workflow using LangChain and Ollama. The Llama 3.2 language model generates answers to user queries, while a text document is loaded and divided into smaller chunks. Hugging Face embeddings convert text into numerical vectors that can be used for semantic search in a RAG system.

Program

Program 1: Generate answers using Ollama

```
from langchain_ollama import ChatOllama
llm = ChatOllama(
   model="llama3.2",
   temperature=0)
response = llm.invoke(
   "Explain Artificial Intelligence in simple words.")
print(response.content)
```

Program 2: Load a text document

```
from langchain_community.document_loaders import TextLoader
loader = TextLoader("AI_Notes.txt")
documents = loader.load()
print(documents)
```

Program 3: Split documents into chunks

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
splitter = RecursiveCharacterTextSplitter(
```

```
    chunk_size=500,
    chunk_overlap=50)
chunks = splitter.split_documents(documents)
print("Number of chunks:", len(chunks))
```

Program 4: Generate text embeddings

```
from langchain_huggingface import HuggingFaceEmbeddings
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2")
vector = embeddings.embed_query(
    "What is deep learning?")
print(vector[:10])
```

Result

Program 1 Result:

```
Artificial Intelligence (AI) is a way to create machines that can think and learn like humans.
Here's a simple explanation:

**What is AI?**

AI is a type of computer science that helps machines make decisions, solve problems, and learn
from data. It's like teaching a computer to recognize pictures, understand speech, and play ga
mes.

**How does AI work?**

AI uses algorithms (like recipes) to process data and make decisions. These algorithms are des
igned to mimic human thought processes, like:

1. **Pattern recognition**: AI can identify patterns in data, like recognizing faces in pictur
es.
2. **Machine learning**: AI can learn from data and improve its performance over time.
3. **Natural Language Processing**: AI can understand and generate human language, like chatti
ng with a virtual assistant.

**Types of AI**

There are several types of AI, including:

1. **Narrow or Weak AI**: Designed to perform a specific task, like playing chess or recognizi
ng faces.
2. **General or Strong AI**: A hypothetical AI that can perform any intellectual task, like hu
man intelligence.
3. **Superintelligence**: An AI that is significantly more intelligent than humans.

**Real-world applications of AI**

AI is used in many areas, such as:

1. **Virtual assistants**: Siri, Alexa, and Google Assistant.
2. **Image recognition**: Self-driving cars, facial recognition, and security systems.
3. **Healthcare**: Medical diagnosis, personalized medicine, and patient care.
4. **Customer service**: Chatbots and automated customer support.

In summary, AI is a powerful technology that helps machines think and learn like humans. It's
used in many areas to improve efficiency, accuracy, and decision-making.
```

Program 2 Result:

```
[Document(metadata={'source': 'AI_Notes.txt'}, page_content='Artificial Intelligence is the fi
eld of building systems that can perform tasks that normally require human intelligence.\n\nMa
chine Learning is a subset of Artificial Intelligence that allows computers to learn patterns
from data.\n\nDeep Learning uses neural networks with multiple layers to learn complex pattern
s.')]
```

Program 3 Result:

```
Number of chunks: 1
```

Program 4 Result:

```
[-0.07785137742757797, -0.033280421048402786, 0.01974339410662651, 0.007226418238133192, -0.01
556931622326374, -0.0220582727342844, -0.03076375648379326, -0.0092751644551754, -0.0262806583
19592476, -0.03895954042673111]
```

Conclusion

The experiment successfully demonstrated text generation using Ollama, document loading and splitting using LangChain, and embedding generation using Hugging Face. These components form the foundation of a RAG system, which can retrieve relevant information from documents and provide context-aware answers. Further development can integrate a vector database such as FAISS or ChromaDB and connect the retrieved document chunks to the language model for question answering.

Exp No: - 6

Date: - 13/10/2026

Implementation of Retrieval-Augmented Generation (RAG) Using LangChain, Ollama, and ChromaDB

Brief Description

This experiment implements a complete Retrieval-Augmented Generation (RAG) pipeline using LangChain, Ollama, and ChromaDB. A sample knowledge document is created and loaded, split into smaller chunks, and converted into embeddings using the Ollama embedding model. The embeddings are stored in ChromaDB for similarity-based retrieval. A retriever fetches relevant document chunks, and the Llama 3.2 language model generates answers using the retrieved context.

Program

```
from pathlib import Path
import chromadb
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_ollama import ChatOllama, OllamaEmbeddings
from langchain_chroma import Chroma
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough
# Step 1: Create a sample knowledge document
knowledge_text = """
Artificial Intelligence (AI) is a field of computer science
that develops systems capable of performing intelligent tasks.
Machine Learning (ML) is a subset of AI. It enables computers
to learn patterns from data and make predictions.
Deep Learning is a subset of machine learning that uses
multi-layer neural networks.
```

```
Natural Language Processing (NLP) enables computers to
understand and generate human language.
Retrieval-Augmented Generation (RAG) combines document
retrieval with a language model to generate answers using
information retrieved from a knowledge base.
LangChain connects language models, prompts, retrievers,
document loaders and other components.
ChromaDB is a vector database that stores embeddings and
retrieves relevant text using vector similarity.
Ollama allows language models and embedding models to
run locally on a computer.
"""
Path("AI_Notes.txt").write_text(
    knowledge_text, encoding="utf-8")
print("STEP 1: Knowledge document created.")
# Step 2: Load the document
loader = TextLoader("AI_Notes.txt", encoding="utf-8")
documents = loader.load()
print("STEP 2: Documents loaded:", len(documents))
# Step 3: Split the document into chunks
splitter = RecursiveCharacterTextSplitter(
    chunk_size=300,
    chunk_overlap=50)
chunks = splitter.split_documents(documents)
print("STEP 3: Number of chunks:", len(chunks))
for i, chunk in enumerate(chunks, start=1):
    print(f"\nChunk {i}:")
    print(chunk.page_content)
# Step 4: Configure the embedding model
embeddings = OllamaEmbeddings(
    model="nomic-embed-text")
print("\nSTEP 4: Embedding model configured.")
# Step 5: Store embeddings in ChromaDB
client = chromadb.PersistentClient(path="./chroma_db")
collection_name = "undergraduate_ai_notes"
existing_collections = [
    collection.name for collection in client.list_collections()]
```

```
if collection_name in existing_collections:
    client.delete_collection(name=collection_name)
vectorstore = Chroma(
    client=client,
    collection_name=collection_name,
    embedding_function=embeddings)
vectorstore.add_documents(chunks)
print("STEP 5: Chunks stored in ChromaDB.")
# Step 6: Create a retriever
retriever = vectorstore.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 3})
print("STEP 6: Retriever created.")
# Step 7: Configure the language model
llm = ChatOllama(
    model="llama3.2",
    temperature=0)
print("STEP 7: Ollama language model configured.")
# Step 8: Create the prompt template
prompt = ChatPromptTemplate.from_template("""
You are an AI teaching assistant.
Answer the question using the supplied context.
If the context does not contain the answer, clearly state
that the provided notes do not contain enough information.
Context:
{context}
Question:
{question}
Answer:
""")
# Step 9: Format retrieved documents
def format_documents(docs):
    return "\n\n".join(
        doc.page_content for doc in docs)
# Step 10: Build the RAG chain
rag_chain = (
    {
```

```
        "context": retriever | format_documents,
        "question": RunnablePassthrough()
    }
    | prompt
    | llm
    | StrOutputParser())
print("STEP 10: Complete RAG chain created.")
# Step 11: Ask a question
question = "What is the difference between AI and ML?"
print("\nUSER QUESTION:", question)
retrieved_docs = retriever.invoke(question)
print("\nRETRIEVED CONTEXT:")
for i, doc in enumerate(retrieved_docs, start=1):
    print(f"\nRetrieved Chunk {i}:")
    print(doc.page_content)
answer = rag_chain.invoke(question)
print("\nFINAL RAG ANSWER:")
print(answer)
# Step 12: Ask additional questions
while True:
    user_question = input(
        "\nAsk another question (or type exit): ").strip()
    if user_question.lower() in {"exit", "quit"}:
        print("RAG demonstration completed.")
        break
    if not user_question:
        print("Please enter a question.")
        continue
    try:
        response = rag_chain.invoke(user_question)
        print("\nANSWER:")
        print(response)
    except Exception as error:
        print("Could not generate the answer:", error)
```

Result

```
STEP 1: Knowledge document created.
STEP 2: Documents loaded: 1
STEP 3: Number of chunks: 4

Chunk 1:
Artificial Intelligence (AI) is a field of computer science
that develops systems capable of performing intelligent tasks.

Machine Learning (ML) is a subset of AI. It enables computers
to learn patterns from data and make predictions.

Chunk 2:
Deep Learning is a subset of machine learning that uses
multi-layer neural networks.

Natural Language Processing (NLP) enables computers to
understand and generate human language.

Chunk 3:
Retrieval-Augmented Generation (RAG) combines document
retrieval with a language model to generate answers using
information retrieved from a knowledge base.

LangChain connects language models, prompts, retrievers,
document loaders and other components.

Chunk 4:
ChromaDB is a vector database that stores embeddings and
retrieves relevant text using vector similarity.

Ollama allows language models and embedding models to
run locally on a computer.

STEP 4: Embedding model configured.
STEP 5: Chunks stored in ChromaDB.
STEP 6: Retriever created.
STEP 7: Ollama language model configured.
STEP 8-10: Complete RAG chain created.

============================================================
USER QUESTION: What is the difference between AI and ML?
============================================================

RETRIEVED CONTEXT:

Retrieved Chunk 1:
Artificial Intelligence (AI) is a field of computer science
that develops systems capable of performing intelligent tasks.

Machine Learning (ML) is a subset of AI. It enables computers
to learn patterns from data and make predictions.

Retrieved Chunk 2:
Deep Learning is a subset of machine learning that uses
multi-layer neural networks.

Natural Language Processing (NLP) enables computers to
understand and generate human language.

Retrieved Chunk 3:
Retrieval-Augmented Generation (RAG) combines document
retrieval with a language model to generate answers using
information retrieved from a knowledge base.

LangChain connects language models, prompts, retrievers,
document loaders and other components.

============================================================
FINAL RAG ANSWER
============================================================
According to the provided context, the difference between AI and ML is not explicitly stated. However, it can be inferred
that AI is a broader field that encompasses various techniques, including ML.

Artificial Intelligence (AI) is a field of computer science that develops systems capable of performing intelligent tasks.
Machine Learning (ML) is a subset of AI that enables computers to learn patterns from data and make predictions.

In other words, AI is the overall field, and ML is a specific technique within that field.
RAG demonstration completed.
```

Conclusion

The experiment successfully demonstrates the implementation of a RAG pipeline using LangChain, Ollama, Hugging Face-style embedding workflows through Ollama, and ChromaDB. The system loads and splits documents, stores embeddings, retrieves relevant information, and generates answers using a local language model. This approach can reduce unsupported responses by grounding answers in retrieved documents, although it does not guarantee that every generated answer is factually correct.